SCR2043 OPERATING SYSTEMS

Name	: CHOH JING YI	Marks
Student ID	: A23CS0296	
Section	: 03	

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. **Download necessary source codes:**

   Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c
wget -O subprocess1.c https://rebrand.ly/subprocess1_c
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. **Compile the source files:**

   Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess
gcc subprocess1.c -o subprocess1
gcc subprocess2.c -o subprocess2
```

3. **Execute the dummy processes:**

   Run all the dummy processes

```
./mainprocess &
```

   Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

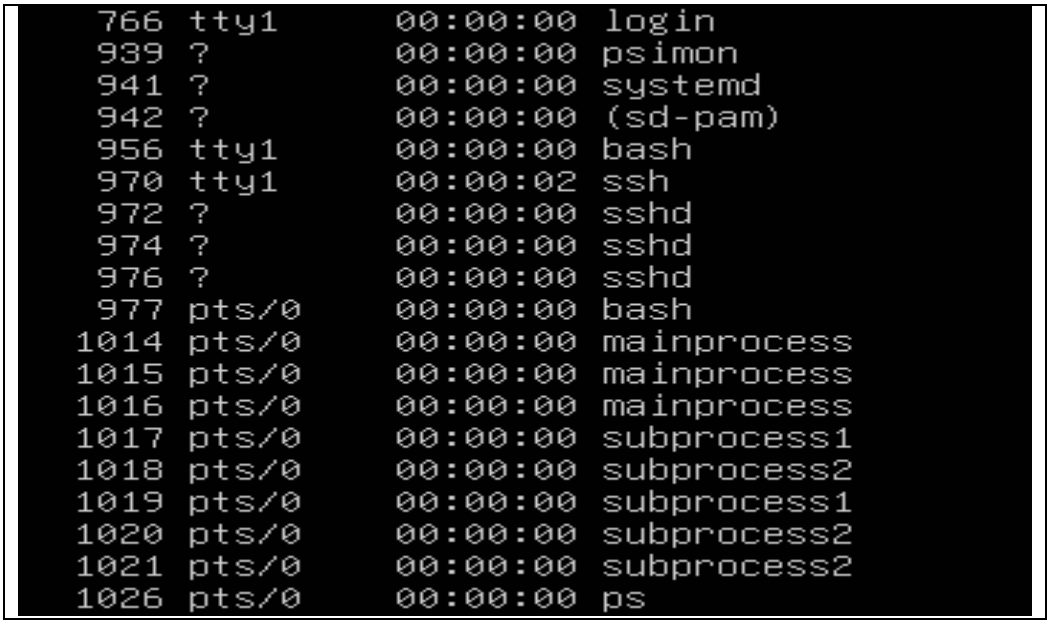
1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

Question 1

Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command
ps -e
Output

```
    155 ?        00:00:00 kworker/R-mpt_p
    156 ?        00:00:00 kworker/R-mpt/0
    157 ?        00:00:00 kworker/0:2H-kblockd
    158 ?        00:00:00 scsi_eh_2
    159 ?        00:00:00 kworker/R-scsi_
    194 ?        00:00:00 kworker/R-raid5
    235 ?        00:00:00 jbd2/sda1-8
    236 ?        00:00:00 kworker/R-ext4-
    309 ?        00:00:00 systemd-journal
    327 ?        00:00:00 kworker/R-kmpat
    328 ?        00:00:00 kworker/R-kmpat
    357 ?        00:00:00 multipathd
    367 ?        00:00:00 systemd-udevd
    378 ?        00:00:00 psimon
    437 ?        00:00:00 irq/18-vmwgfx
    438 ?        00:00:00 kworker/R-ttm
    483 ?        00:00:00 jbd2/sda16-8
    484 ?        00:00:00 kworker/R-ext4-
    517 ?        00:00:00 systemd-network
    527 ?        00:00:00 systemd-resolve
    529 ?        00:00:00 systemd-timesyn
    643 ?        00:00:00 dbus-daemon
    647 ?        00:00:00 polkitd
    653 ?        00:00:00 systemd-logind
    656 ?        00:00:00 udisksd
    670 ?        00:00:00 kworker/u5:3-events_unbound
    671 ?        00:00:00 rsyslogd
    681 ?        00:00:00 unattended-upgr
    710 ?        00:00:00 ModemManager
    744 ?        00:00:00 cron
    766 tty1     00:00:00 login
```



Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command
ps -ef |grep -E ‘inprocess|subprocess’
Output

```
chohjingyi@secr2043:~$ ps -ef |grep -E 'inprocess|subprocess'
chohjin+    1014     977  0 08:56 pts/0    00:00:00 ./mainprocess
chohjin+    1015    1014  0 08:56 pts/0    00:00:00 ./mainprocess
chohjin+    1016    1014  0 08:56 pts/0    00:00:00 ./mainprocess
chohjin+    1017    1015  0 08:56 pts/0    00:00:00 ./subprocess1
chohjin+    1018    1016  0 08:56 pts/0    00:00:00 ./subprocess2
chohjin+    1019    1015  0 08:56 pts/0    00:00:00 ./subprocess1
chohjin+    1020    1016  0 08:56 pts/0    00:00:00 ./subprocess2
chohjin+    1021    1016  0 08:56 pts/0    00:00:00 ./subprocess2
chohjin+    1044     977  0 09:09 pts/0    00:00:00 grep --color=auto -E inprocess|subprocess
chohjingyi@secr2043:~$
```

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command
ps -ef |grep -E 'subprocess'
Output

```
chohjingyi@secr2043:~$ ps -ef |grep -E 'subprocess'
chohjin+    1017    1015  0 08:56 pts/0    00:00:00 ./subprocess1
chohjin+    1018    1016  0 08:56 pts/0    00:00:00 ./subprocess2
chohjin+    1019    1015  0 08:56 pts/0    00:00:00 ./subprocess1
chohjin+    1020    1016  0 08:56 pts/0    00:00:00 ./subprocess2
chohjin+    1021    1016  0 08:56 pts/0    00:00:00 ./subprocess2
chohjin+    1061     977  0 09:16 pts/0    00:00:00 grep --color=auto -E subprocess
```

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)
- Command (`cmd`)

Command
ps -eo pid,user,pcpu,pmem,cmd |grep -E 'subprocess'
Output

```
chohjingyi@secr2043:~$ ps -eo pid,user,pcpu,pmem,cmd |grep -E 'subprocess'
   1017 chohjin+  0.0  0.1 ./subprocess1
   1018 chohjin+  0.0  0.1 ./subprocess2
   1019 chohjin+  0.0  0.1 ./subprocess1
   1020 chohjin+  0.0  0.1 ./subprocess2
   1021 chohjin+  0.0  0.1 ./subprocess2
   1050 chohjin+  0.0  0.1 grep --color=auto -E subprocess
chohjingyi@secr2043:~$
```

Question 5

Building on the ps command used in Question 4, can you add an option to sort the listed processes by their memory usage (`pmem`)?

Command
ps -eo pid,user,pcpu,pmem,cmd –sort=pmem |grep -E 'subprocess'
Output

```
chohjingyi@secr2043:~$ ps -eo pid,user,pcpu,pmem,cmd --sort=pmem |grep -E 'subprocess'
   1017 chohjin+  0.0  0.1 ./subprocess1
   1018 chohjin+  0.0  0.1 ./subprocess2
   1019 chohjin+  0.0  0.1 ./subprocess1
   1020 chohjin+  0.0  0.1 ./subprocess2
   1021 chohjin+  0.0  0.1 ./subprocess2
   1054 chohjin+  0.0  0.1 grep --color=auto -E subprocess
```

Question 6

Construct a command using `ps`, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command
ps --forest -C mainprocess -C subprocess1 -C subprocess2
Output

```
chohjingyi@secr2043:~$ ps --forest -C mainprocess -C subprocess1 -C subprocess2
    PID TTY          TIME CMD
   1014 pts/0    00:00:00 mainprocess
   1015 pts/0    00:00:00  \_ mainprocess
   1017 pts/0    00:00:00  |   \_ subprocess1
   1019 pts/0    00:00:00  |   \_ subprocess1
   1016 pts/0    00:00:00  \_ mainprocess
   1018 pts/0    00:00:00      \_ subprocess2
   1020 pts/0    00:00:00      \_ subprocess2
   1021 pts/0    00:00:00      \_ subprocess2
```

Question 7

Use `pstree` command with option that show the number of threads to each process.

Command
pstree -c -s 1014
Output

```
chohjingyi@secr2043:~$ pstree -c -s 1014
systemd---sshd---sshd---sshd---bash---mainprocess-+-mainprocess-+-subprocess1
                                                   |             `-subprocess1
                                                   `-mainprocess-+-subprocess2
                                                                 |-subprocess2
                                                                 `-subprocess2
```

Question 8

Use `renice` command to change priority level of one of process “subprocess1”.

Command
sudo renice -5 1017
Output

```
chohjingyi@secr2043:~$ ps -o pid,nice,comm -C subprocess1
    PID  NI COMMAND
   1017   0 subprocess1
   1019   0 subprocess1
chohjingyi@secr2043:~$ sudo renice -5 1017
[sudo] password for chohjingyi:
1017 (process ID) old priority 0, new priority -5
```

Question 9

Terminate all running processes with the name “mainprocess”.

Command
killall -15 mainprocess
Output

```
chohjingyi@secr2043:~$ killall -15 mainprocess
Main process (ID: 1015) received signal: 15. Terminating...
Main process (ID: 1016) received signal: 15. Terminating...
chohjingyi@secr2043:~$ Main process (ID: 1014) received signal: 15. Terminating...
```

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

```
import os

def child_process():
    print(f"[Child] PID: {os.getpid()}, Parent PID: {os.getppid()}")

def parent_process(child_pid):
    print(f"[Parent] Created child with PID: {child_pid}")
    pid, status = os.wait()  # Wait for the child to finish
    print(f"[Parent] Child with PID {pid} finished with status {status}")

def main():
    pid = os.fork()

    if pid == 0:
        # Child process
        child_process()
        os._exit(0)  # Exit child process explicitly
    else:
        # Parent process
        parent_process(pid)

if __name__ == "__main__":
    main()
```

Output: